Code Walkthrough

Soncini Curie Shielding and RKB Long-Distance Analysis

Goal: explain the program in the same order in which it is written and executed: what each block does, why it is implemented that way, what can be changed, and how to read the numerical output.

1 What the program does

The script post-processes two ReSpect output files. It reads the total and decomposed EPR g tensor and hyperfine-coupling (HFCC) tensors, then evaluates the Curie contribution to the isotropic paramagnetic NMR shift. It also prints the full Curie shielding tensor, point-dipole/long-distance quantities, RKB-inspired long-distance tensor residuals, all 3×5 g-part$\times A$-part products, and writes text/CSV/cube files.

The most important terminology is

$$\boxed{\boldsymbol{\sigma}^{\mathrm{Curie}} \text{ is a shielding tensor}}, \qquad \boxed{d_* \equiv \delta_* \text{ is a scalar shift in ppm}}.$$

Thus `d_total` is not a tensor. It is the scalar Curie shift

$$\delta_{\mathrm{Curie}} = -10^6 \, \frac{1}{3} \, \mathrm{Tr}\, \boldsymbol{\sigma}^{\mathrm{Curie}}.$$

The program does not construct a complete experimental NMR chemical shift; it evaluates the Curie paramagnetic part from the ReSpect g and A tensors.

The execution flow is

$$\text{Settings} \rightarrow \text{read files} \rightarrow \text{parse tensors} \rightarrow \text{physics} \rightarrow \text{checks} \rightarrow \text{Tables 1–4} \rightarrow \text{TXT/CSV/CUBE}.$$

2 Imports, settings and constants

The first executable block contains all parameters that a normal user should change. The rest of the code can usually be left untouched.

```
from __future__ import annotations
import csv, io, math, re
from contextlib import redirect_stdout
from dataclasses import dataclass
from pathlib import Path, PureWindowsPath
from typing import Dict, List, Optional, Tuple
import numpy as np


# ---------------------------- SETTINGS --------------------------------------
@dataclass(frozen=True)
class Settings:
    # Input files: same folder as this Python script.
    gt_file: str = "2c-gt.out_gt"
    hfs_file: str = "2c-hfs.out_hfcc"

    # All generated files go into one subfolder next to the script.
    output_dir: str = "pnmr_output"

    temperature_K: float = 298.15
    spin_S: float = 0.5
    center_atom: int = 1
    near_A: float = 4.0
    far_A: float = 8.0

    write_txt: bool = True
    txt_file: str = "pnmr_results.txt"

    write_csv: bool = True
    csv_file: str = "pnmr_results.csv"

    write_cube: bool = True
    cube_file: str = "pcs_field.cube"
    cube_spacing_A: float = 0.4
    cube_padding_A: float = 4.0
    cube_cutoff_A: float = 1.0


S = Settings()

# SI constants + a.u. constants used only by the SI/a.u. check.
mu0 = 4 * math.pi * 1e-7
muB = 9.2740100783e-24
muN = 5.0507837461e-27
kB = 1.380649e-23
h = 6.62607015e-34
Eh = 4.3597447222071e-18
c_au = 137.035999084
mp_me = 1836.15267343
A_per_m = 1e-10
bohr_per_A = 1.889726124565062

beta_si = 1 / (kB * S.temperature_K)
beta_au = Eh / (kB * S.temperature_K)
spin_factor = S.spin_S * (S.spin_S + 1) / 3
_FLOAT = r"[-+]?\d+(?:\.\d*)?(?:[Ee][-+]?\d+)?"
```

What can be changed here

Setting	Meaning
`gt_file`, `hfs_file`	Input filenames. Normally the files are placed in the same directory as the Python script.
`output_dir`	Directory created next to the script for all generated files.
`temperature_K`	Temperature entering $\beta = 1/(k_B T)$. Curie contributions scale approximately as $1/T$ if g and A are fixed.
`spin_S`	Effective spin. The code uses $S(S+1)/3$.
`center_atom`	Atom used as the paramagnetic centre for all R-dependent PDA/LD quantities, Table 3 and the cube. It does not change `d_total`, which comes directly from ReSpect g and A.
`near_A`, `far_A`	Only classification thresholds for the labels near/mid/far.
`write_txt/csv/cube`	Turn individual output files on/off.
`cube_spacing_A`	Cube resolution; smaller spacing produces a finer and larger grid.
`cube_cutoff_A`	Region around the singular point-dipole centre that is suppressed.

The physical constants are stored once in SI. A few atomic-unit constants are present only for the independent SI/a.u. consistency check. The regular expression `_FLOAT` is used by the parser to recognize ordinary and scientific-notation numbers.

3 Locating files and reading 3×3 matrices

The next block is infrastructure rather than physics:

```
# ---------------------------- PARSING -------------------------------------
def script_dir() -> Path:
    return Path(__file__).resolve().parent


def resolve_input_file(name: str) -> Path:
    p = Path(name)
    if p.is_absolute() and p.exists():
        return p

    candidate = script_dir() / PureWindowsPath(name).name
    if candidate.exists():
        return candidate

    raise FileNotFoundError(
        f"Input file not found: {candidate}\n"
        f"Put '{PureWindowsPath(name).name}' in the same folder as this script."
    )


def output_path(name: str) -> Path:
    folder = script_dir() / S.output_dir
    folder.mkdir(parents=True, exist_ok=True)
    return folder / Path(name).name


def matrix3(lines: List[str]) -> np.ndarray:
    rows = []
    for line in lines:
        x = [float(v) for v in re.findall(_FLOAT, line)]
        if len(x) >= 3:
            rows.append(x[:3])
        if len(rows) == 3:
            return np.array(rows)
    raise RuntimeError("3x3 matrix not found")
```

`script_dir()` returns the location of the Python file itself. This is why the script works even if it is launched from a different working directory. `resolve_input_file()` first accepts a valid absolute path; otherwise it looks for the basename next to the script. `output_path()` creates the output directory only when it is needed.

`matrix3()` is a small generic parser: it searches lines for numbers, takes the first three numerical values from three suitable lines, and converts them to a NumPy 3×3 array. Keeping this logic in one helper prevents repetition in every tensor parser.

4 Parsing the ReSpect files

4.1 Geometry, adjusted c, gauge origin and g tensor

```
def parse_geometry(text: str) -> Dict[int, Tuple[str, np.ndarray]]:
    lines = text.splitlines()
    start = next(i for i, s in enumerate(lines) if "Molecular geometry [A]" in s)
    pat = re.compile(rf"^\s*([A-Za-z]{{1,3}})\s+(\d+)\s+{_FLOAT}\s+({_FLOAT})\s+({_FLOAT})\s+({_FLOAT})\s*$")
    out = {}
    for line in lines[start + 1:]:
        if out and not line.strip():
            break
        if m := pat.match(line):
            out[int(m.group(2))] = (m.group(1), np.array([float(m.group(i)) for i in (3, 4, 5)]))
    if not out:
        raise RuntimeError("Geometry not found")
    return out


def parse_c_adjust(text: str) -> Tuple[Optional[float], Optional[float]]:
    a = re.search(r"Speed of light is adjusted\s+([-+0-9.Ee]+)\s+times", text)
    c = re.search(r"Speed of light is now:\s+([-+0-9.Ee]+)", text)
    return (float(a.group(1)) if a else None, float(c.group(1)) if c else None)


def parse_gauge_origin(text: str) -> Optional[np.ndarray]:
    m = re.search(r"GO\s*=\s*(" + _FLOAT + r")\s+(" + _FLOAT + r")\s+(" + _FLOAT + r")", text)
    return np.array([float(m.group(i)) for i in (1, 2, 3)]) if m else None


def parse_g_tensor(text: str) -> Dict[str, np.ndarray]:
    keys = {"Spin-Zeeman term": "SZ", "Orbital-Zeeman term": "OZ",
            "Relativistic term": "Rel", "g(B,S)": "total"}
```

```
    lines, out = text.splitlines(), {}
    for i, line in enumerate(lines):
        label = line.strip().rstrip(":")
        if label in keys:
            out[keys[label]] = matrix3(lines[i + 1:i + 20])
    if "total" not in out:
        raise RuntimeError("g tensor not found")
    return out
```

`parse_geometry()` locates the `Molecular geometry [A]` block and builds

$$\text{atom number} \mapsto (\text{element}, \mathbf{r}).$$

The coordinates are kept in Å because that is the natural geometry unit of the ReSpect output and the long-distance functions explicitly convert R to metres only where SI is needed.

`parse_c_adjust()` and `parse_gauge_origin()` read metadata. The gauge origin is printed but is *not* used as the origin of the long-distance vector $\mathbf{R}$.

`parse_g_tensor()` searches for the exact ReSpect labels and returns a dictionary containing

$$g_{\text{total}}, \quad g_{\text{SZ}}, \quad g_{\text{OZ}}, \quad g_{\text{Rel}}.$$

The intended decomposition is

$$\boxed{g = g_{\text{SZ}} + g_{\text{OZ}} + g_{\text{Rel}}}.$$

4.2 HFCC blocks

```
def parse_hfcc(text: str) -> Dict[int, Dict]:
    lines = text.splitlines()
    starts = [(i, int(m.group(1))) for i, line in enumerate(lines)
              if (m := re.search(r"HFCC FOR NUCLEUS\s*#\s*(\d+)\s*=", line))]
    tags = {"A": r"a\(I,S\) \[MHz\]:", "FC": r"Fermi-contact term \(FC\):",
            "PSO": r"Paramagnetic spin orbit term \(PSO\):",
            "SD": r"Spin dipolar term \(SD\):",
            "R1": r"First relativistic term \(R1\):",
            "R2": r"Second relativistic term \(R2\):"}
    if not starts:
        raise RuntimeError("HFCC blocks not found")

    out = {}
    for b, (i0, idx) in enumerate(starts):
        block = lines[i0:starts[b + 1][0] if b + 1 < len(starts) else len(lines)]
        txt = "\n".join(block)
        rec = {}
        if m := re.search(r"g-factor\s*=\s*([-+0-9.Ee]+)", txt):
            rec["gN"] = float(m.group(1))
        for name, pat in tags.items():
            for j, line in enumerate(block):
                if re.match(r"\s*" + pat + r"\s*$", line):
                    rec[name] = matrix3(block[j + 1:j + 15])
                    break
        out[idx] = rec
    return out
```

`parse_hfcc()` first identifies every `HFCC FOR NUCLEUS #...` block. Each nucleus is processed separately, which prevents the parser from accidentally taking a matrix from the following nucleus. It extracts the nuclear g factor and the ReSpect decomposition

$$\boxed{A = A_{\text{FC}} + A_{\text{PSO}} + A_{\text{SD}} + A_{\text{R1}} + A_{\text{R2}}}.$$

If a future ReSpect version changes a printed heading, this `tags` dictionary is the main place that must be updated.

5 Core Curie-shift physics

This is the most important part of the script.

```
# ---------------------------- PHYSICS ---------------------------------------
def iso(M: np.ndarray) -> float:
    return float(np.trace(M) / 3)


def shift_ppm(g: np.ndarray, A_MHz: np.ndarray, gN: float) -> float:
    A_J = h * 1e6 * A_MHz
    sigma_iso = -(muB / (gN * muN)) * beta_si * spin_factor * iso(g @ A_J.T)
    return -1e6 * sigma_iso


def shielding_tensor_ppm(g: np.ndarray, A_MHz: np.ndarray, gN: float) -> np.ndarray:
    """Full Curie shielding tensor multiplied by 1e6, i.e. printed in ppm."""
    A_J = h * 1e6 * A_MHz
    sigma = -(muB / (gN * muN)) * beta_si * spin_factor * (g @ A_J.T)
    return 1e6 * sigma


def split_iso_ani(M: np.ndarray) -> Tuple[float, np.ndarray]:
    x = iso(M)
    return x, M - x * np.eye(3)


def shift_iso_ani(g: np.ndarray, A: np.ndarray, gN: float) -> Tuple[float, float]:
    gi, ga = split_iso_ani(g)
    ai, aa = split_iso_ani(A)
    return shift_ppm(gi * np.eye(3), ai * np.eye(3), gN), shift_ppm(ga, aa, gN)


def shift_ppm_au(g: np.ndarray, A_MHz: np.ndarray, gN: float) -> float:
    A_Eh = h * 1e6 * A_MHz / Eh
    gamma_N = gN / (2 * c_au * mp_me)
    sigma_iso = -beta_au / (2 * c_au * gamma_N) * spin_factor * iso(g @ A_Eh.T)
    return -1e6 * sigma_iso
```

5.1 `iso`

For any 3×3 tensor,

$$\mathrm{iso}(M) = \frac{1}{3}\,\mathrm{Tr}\,M.$$

This is the rotational average relevant to an isotropic solution.

5.2 `shift_ppm`: shielding → scalar shift

The theoretical Curie shielding tensor is

$$\boxed{\boldsymbol{\sigma}^{\mathrm{Curie}} = -\frac{\mu_B}{g_N \mu_N}\,\beta\,\frac{S(S+1)}{3}\,g A^{\mathrm{T}}}.$$

ReSpect provides A in MHz, so the code first converts

$$A[\mathrm{MHz}] \to h10^6 A[\mathrm{J}].$$

Then it takes the isotropic trace and returns

$$\boxed{\delta_{\mathrm{Curie}}[\mathrm{ppm}] = -10^6\,\sigma_{\mathrm{iso}}}.$$

The function is deliberately written for arbitrary g and A arguments. Because the expression is bilinear, the same routine can evaluate the total shift, individual ReSpect components, and all Table-4 products.

5.3 `shielding_tensor_ppm`

This function keeps the complete 3×3 matrix instead of taking the trace. It returns $10^6\boldsymbol{\sigma}^{\mathrm{Curie}}$, so its matrix elements are conveniently printed on a ppm scale. The consistency relation is

$$\boxed{\delta_{\mathrm{Curie}} = -\frac{1}{3}\,\mathrm{Tr}\big(10^6\boldsymbol{\sigma}^{\mathrm{Curie}}\big)}.$$

5.4 Isotropic/anisotropic split and a.u. check

`split_iso_ani()` constructs

$$M = M_{\mathrm{iso}} I + M_{\mathrm{ani}}, \qquad \mathrm{Tr}\,M_{\mathrm{ani}} = 0.$$

Therefore

$$\delta_{\mathrm{total}} = \delta_{\mathrm{iso}} + \delta_{\mathrm{ani}}.$$

Here `d_ani` is a scalar anisotropic–anisotropic contribution to the isotropic Curie shift; it is not a CSA tensor. `shift_ppm_au()` implements the same final scalar quantity in atomic units and is used only as a unit/prefactor check.

6 Long-distance, point-dipole and RKB diagnostics

```
def susceptibility(g: np.ndarray) -> np.ndarray:
    return mu0 * muB**2 * beta_si * spin_factor * (g @ g.T)


def ld_shift_ppm(chi: np.ndarray, R_A: np.ndarray) -> float:
    R = np.linalg.norm(R_A)
    if R < 1e-9:
        return float("nan")
    u = R_A / R
    D = 3 * np.outer(u, u) - np.eye(3)
    return 1e6 * iso(chi @ D) / (4 * math.pi * (R * A_per_m)**3)


def pda_A_MHz(g: np.ndarray, R_A: np.ndarray, gN: float) -> np.ndarray:
    R = np.linalg.norm(R_A)
    if R < 1e-9:
        return np.full((3, 3), np.nan)
    u = R_A / R
    D = 3 * np.outer(u, u) - np.eye(3)
    pref = mu0 * (gN * muN) * muB / (4 * math.pi * (R * A_per_m)**3)
    return pref * (D @ g) / (h * 1e6)


def principal_g(g: np.ndarray) -> np.ndarray:
    return np.sort(np.sqrt(np.clip(np.linalg.eigvalsh(g @ g.T), 0, None)))


def cross_terms(gp: Dict[str, np.ndarray], rec: Dict, gN: float) -> Dict[Tuple[str, str], float]:
    return {(gn, an): shift_ppm(gp[gn], rec[an], gN)
            for gn in ("SZ", "OZ", "Rel") if gn in gp
            for an in ("FC", "PSO", "SD", "R1", "R2") if an in rec}


def rkb_check(gp: Dict[str, np.ndarray], rec: Dict, R: np.ndarray, gN: float) -> Optional[Dict]:
    if np.linalg.norm(R) < 1e-9 or any(k not in rec for k in ("FC", "PSO", "SD", "R1", "R2")) \
       or any(k not in gp for k in ("SZ", "OZ", "Rel")):
        return None

    Arel = rec["R1"] + rec["R2"]
    pairs = {
        "SD": (rec["SD"], pda_A_MHz(gp["SZ"], R, gN)),
        "PSO": (rec["PSO"], pda_A_MHz(gp["OZ"], R, gN)),
        "REL": (Arel, pda_A_MHz(gp["Rel"], R, gN)),
        "COMB": (rec["PSO"] + Arel,
                 pda_A_MHz(gp["OZ"], R, gN) + pda_A_MHz(gp["Rel"], R, gN)),
    }
    out = {"R_A": float(np.linalg.norm(R)), "FC_norm": float(np.linalg.norm(rec["FC"]))}
    for name, (a, p) in pairs.items():
```

```
        n = np.linalg.norm(a)
        out[f"{name}_rel"] = float(np.linalg.norm(a - p) / n) if n > 1e-12 else float("nan")
    return out
```

6.1 Susceptibility and long-distance PCS

The Curie susceptibility is

$$\boxed{\chi = \mu_0 \mu_B^2 \beta \frac{S(S+1)}{3} g g^{\mathrm{T}}}.$$

For $\mathbf{R}$ from `center_atom` to a nucleus,

$$D = 3\hat{\mathbf{R}}\hat{\mathbf{R}}^{\mathrm{T}} - I,$$

and the code evaluates

$$\boxed{\delta_{\mathrm{LD}} = 10^6 \frac{\mathrm{iso}(\chi D)}{4\pi R^3}}.$$

At $R = 0$ the point-dipole formula is undefined, so the code returns `nan`.

6.2 Point-dipole HFC tensor

`pda_A_MHz()` constructs

$$A_{\mathrm{PDA}} = \frac{\mu_0 (g_N \mu_N) \mu_B}{4\pi R^3 h 10^6} \, Dg$$

in MHz and then the ordinary `shift_ppm()` function is applied to it. Hence `d_PDA` and `d_LD` are two algebraically equivalent routes to the same point-dipole scalar quantity. Their agreement tests units, signs and geometry; it is not an independent proof that the finite-distance ReSpect A tensor is point-dipolar.

6.3 Principal g values

For a possibly nonsymmetric g, the program uses singular values,

$$g_i = \sqrt{\lambda_i(g g^{\mathrm{T}})},$$

which is why `principal_g()` diagonalizes $g g^{\mathrm{T}}$ rather than simply diagonalizing the printed g matrix.

6.4 All 15 products

`cross_terms()` evaluates

$$\delta(g_i, A_j), \qquad g_i \in \{SZ, OZ, Rel\}, \quad A_j \in \{FC, PSO, SD, R1, R2\}.$$

There are $3 \times 5 = 15$ values and, if the printed decompositions reconstruct the totals,

$$\boxed{\sum_{ij} \delta(g_i, A_j) = \delta_{\mathrm{total}}}.$$

No matched/mismatched physical interpretation is imposed.

6.5 `rkb_check`

The function forms four tensor comparisons:

$$SD \leftrightarrow PDA[SZ], \quad PSO \leftrightarrow PDA[OZ], \quad R1 + R2 \leftrightarrow PDA[Rel],$$

and a combined PSO+relativistic comparison. It reports the normalized Frobenius residual

$$\boxed{\mathrm{rel} = \frac{\|A_{\mathrm{actual}} - A_{\mathrm{pred}}\|_F}{\|A_{\mathrm{actual}}\|_F}}.$$

A value near zero means close tensors; a value of order one means the difference is comparable to the tensor itself. SD/SZ is the cleanest direct long-distance correspondence. PSO and the relativistic block should be treated as diagnostics because the analytical theory contains gauge/rearrangement subtleties.

7 How the program prints the results

7.1 Tables 1 and 2, raw product and shielding tensor

```
# ---------------------------- OUTPUT -----------------------------------------
def print_tables12(rows: List[Dict]) -> None:
    meta = f"{'At':>3} {'El':>2} {'R/A':>6} {'lbl':>6}"

    print("\nTABLE 1 [ppm]")
    head = meta + f" {'Aiso':>9} {'d_total':>11} {'d_nonFC':>10} {'d_iso':>10} {'d_ani':>10} {'d_PDA':>10} {'d_LD':>10}"
    print(head); print("-" * len(head))
    for r in rows:
        print(f"{r['Atom']:3d} {r['El']:>2} {r['R_A']:6.3f} {r['label']:>6} {r['Aiso_MHz']:9.3f} "
              f"{r['d_total']:11.4g} {r['d_nonFC']:10.3g} {r['d_iso']:10.3g} {r['d_ani']:10.3g} "
              f"{r['d_PDA']:10.3g} {r['d_LD']:10.3g}")

    ac = [c for c in ("d_FC", "d_PSO", "d_SD", "d_R1", "d_R2") if any(c in r for r in rows)]
    gc = [c for c in ("d_gSZ", "d_gOZ", "d_gRel") if any(c in r for r in rows)]
    print("\nTABLE 2 [ppm]")
    head = meta + "".join(f" {c[2:]:>9}" for c in ac + gc)
    print(head); print("-" * len(head))
    for r in rows:
        print(f"{r['Atom']:3d} {r['El']:>2} {r['R_A']:6.3f} {r['label']:>6}" +
              "".join(f" {r.get(c, float('nan')):9.3g}" for c in ac + gc))


def print_gAT(rows: List[Dict], mats: Dict[int, np.ndarray],
              sigmas: Dict[int, np.ndarray]) -> None:
    print("\ng @ A_total^T [MHz-like] + Curie shielding sigma [ppm]")
    for r in rows:
        idx = r["Atom"]
        M = mats[idx]
        sigma = sigmas[idx]

        print(f"\nAtom #{idx} {r['El']} R={r['R_A']:.4f} A")

        print(" g @ A_total^T:")
        for x in M:
            print(" " + " ".join(f"{v:14.6e}" for v in x))
        print(f" Tr/3={iso(M):.6e}")

        print(" sigma_Curie [ppm]:")
        for x in sigma:
            print(" " + " ".join(f"{v:14.6e}" for v in x))
        print(f" sigma_iso={iso(sigma):.6e} ppm -> delta={r['d_total']:.4g} ppm")
```

Table 1 is the main per-nucleus summary. The columns mean:

- `Aiso`: $\mathrm{Tr}\, A/3$ in MHz;
- `d_total`: total scalar Curie shift;
- `d_nonFC`: shift from $A - A_{FC}$;
- `d_iso`, `d_ani`: isotropic–isotropic and anisotropic–anisotropic scalar parts;
- `d_PDA`, `d_LD`: two equivalent point-dipole/long-distance routes.

Table 2 applies the same `shift_ppm()` function to the ReSpect components. FC–R2 use the total g with one A part; gSZ–gRel use one g part with the total A.

`print_gAT()` deliberately prints two matrices. The first is the raw product gA^{T} ("MHz-like"). The second is the actual Curie shielding matrix $10^6\boldsymbol{\sigma}^{\mathrm{Curie}}$. The last line shows the direct relation between the matrix trace and the scalar δ.

7.2 Tables 3 and 4

```
def print_rkb(rows: List[Dict]) -> None:
    if not rows:
        return
    rows = sorted(rows, key=lambda r: r["R_A"])
    print("\nTABLE 3 -- LD tensor residuals rel=||A-PDA||/||A||")
    head = f"{'At':>3} {'El':>2} {'R/A':>6} {'lbl':>6} {'||FC||':>9} {'SD/SZ':>9} {'PSO/OZ':>9} {'R1R2/Rel':>10} {'COMB':>9}"
    print(head); print("-" * len(head))
    for r in rows:
        print(f"{r['Atom']:3d} {r['El']:>2} {r['R_A']:6.3f} {r['label']:>6} {r['FC_norm']:9.3g} "
              f"{r['SD_rel']:9.3g} {r['PSO_rel']:9.3g} {r['REL_rel']:10.3g} {r['COMB_rel']:9.3g}")
    far = [r for r in rows if r["label"] == "far"] or rows
    av = lambda k: sum(r[k] for r in far) / len(far)
    print(f"far avg: SD={av('SD_rel'):.3g} PSO={av('PSO_rel'):.3g} R1R2={av('REL_rel'):.3g} COMB={av('COMB_rel'):.3g}")


def print_cross(rows: List[Dict], crosses: Dict[int, Dict[Tuple[str, str], float]]) -> None:
    print("\nTABLE 4 -- shift(g_part, A_part) [ppm]")
    anames, gnames = ("FC", "PSO", "SD", "R1", "R2"), ("SZ", "OZ", "Rel")
    for r in rows:
        c = crosses.get(r["Atom"], {})
        if not c:
            continue
        print(f"\nAtom #{r['Atom']} {r['El']} R={r['R_A']:.4f} A")
        print(" " + "".join(f"{a:>12}" for a in anames))
        for g in gnames:
            print(f"{g:>6} " + "".join(f"{c.get((g, a), float('nan')):12.4g}" for a in anames))
        print(f"sum={sum(c.values()):.6g} d_total={r['d_total']:.6g}")
```

Table 3 sorts nuclei by distance and prints the residuals described above. The "far avg" is only a descriptive mean over nuclei carrying the geometric `far` label.

Table 4 prints the full 3×5 grid for every nucleus. The final `sum=` line is the easiest visual check that all 15 bilinear contributions reconstruct `d_total`.

8 Output files

```
def write_text_report(rows, mats, sigmas, rkb_rows, crosses,
                      g, go, c_scale, c_now, checks) -> Path:
    """Write the same results as a compact ReSpect-like text report."""
    path, buf = output_path(S.txt_file), io.StringIO()
    with redirect_stdout(buf):
        print("=" * 79)
        print(" pNMR CURIE-SHIFT POST-PROCESSING RESULTS")
        print("=" * 79)
        print(f"T={S.temperature_K:g} K S={S.spin_S:g} center_atom={S.center_atom}")
        pg = principal_g(g)
        print(f"g_iso={iso(g):.6f} g_principal={pg[0]:.6f},{pg[1]:.6f},{pg[2]:.6f}")
        if go is not None:
            print(f"GO[A]={go[0]:.4f},{go[1]:.4f},{go[2]:.4f}")
        if c_scale is not None:
```

```
            print(f"c_scale={c_scale:g} c={c_now:g} a.u.")
        print("checks[ppm]: " + " ".join(f"{k}={v:.2e}" for k, v in checks.items()))
        print_tables12(rows)
        print_gAT(rows, mats, sigmas)
        print_rkb(rkb_rows)
        print_cross(rows, crosses)
        print("\n" + "=" * 79)
        print(" END OF pNMR CURIE-SHIFT RESULTS")
        print("=" * 79)
    path.write_text(buf.getvalue(), encoding="utf-8")
    return path

def write_csv(rows: List[Dict]) -> Path:
    path = output_path(S.csv_file)
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)
    return path


def write_cube(atoms: Dict[int, Tuple[str, np.ndarray]], center: np.ndarray, chi: np.ndarray) -> Dict:
    symbols = ("X H He Li Be B C N O F Ne Na Mg Al Si P S Cl Ar K Ca Sc Ti V Cr Mn Fe Co Ni Cu Zn "
               "Ga Ge As Se Br Kr Rb Sr Y Zr Nb Mo Tc Ru Rh Pd Ag Cd In Sn Sb Te I Xe").split()
    Z = {e: i for i, e in enumerate(symbols)}

    xyz = np.array([v[1] for v in atoms.values()])
    origin = xyz.min(0) - S.cube_padding_A
    dims = np.ceil((xyz.max(0) + S.cube_padding_A - origin) / S.cube_spacing_A).astype(int) + 1
    nx, ny, nz = map(int, dims)

    xs = origin[0] + S.cube_spacing_A * np.arange(nx)
    ys = origin[1] + S.cube_spacing_A * np.arange(ny)
    zs = origin[2] + S.cube_spacing_A * np.arange(nz)
    X, Y, Zc = np.meshgrid(xs, ys, zs, indexing="ij")
    R = np.stack((X-center[0], Y-center[1], Zc-center[2]), axis=-1)
    rn = np.linalg.norm(R, axis=-1)
    rs = np.where(rn < S.cube_cutoff_A, np.inf, rn)
    u = R / rs[..., None]
    q = np.einsum("...i,ij,...j->...", u, chi, u)
    values = (3*q - np.trace(chi)) / (12*math.pi*(rs*A_per_m)**3) * 1e6

    ob, step = origin * bohr_per_A, S.cube_spacing_A * bohr_per_A
    path = output_path(S.cube_file)
    with open(path, "w") as f:
        f.write("Long-distance pseudocontact shift field delta_pc(r) [ppm]\n\n")
        f.write(f"{len(atoms):5d}{ob[0]:13.6f}{ob[1]:13.6f}{ob[2]:13.6f}\n")
        f.write(f"{nx:5d}{step:13.6f}{0:13.6f}{0:13.6f}\n")
        f.write(f"{ny:5d}{0:13.6f}{step:13.6f}{0:13.6f}\n")
        f.write(f"{nz:5d}{0:13.6f}{0:13.6f}{step:13.6f}\n")
        for el, pos in atoms.values():
            z, p = Z[el], pos * bohr_per_A
            f.write(f"{z:5d}{float(z):13.6f}{p[0]:13.6f}{p[1]:13.6f}{p[2]:13.6f}\n")
        flat = values.ravel()
        for i in range(0, len(flat), 6):
            f.write("".join(f"{v:13.5e}" for v in flat[i:i+6]) + "\n")
    return {"path": path, "nx": nx, "ny": ny, "nz": nz,
            "min": float(values.min()), "max": float(values.max())}
```

The text report reuses the same print functions through `redirect_stdout`; therefore the human-readable file and terminal output cannot silently drift apart. The CSV stores all scalar values, all nine elements of gA^{T}, Table-3 residuals and all 15 Table-4 products.

The cube is a different kind of output: it evaluates the pure long-distance field on a 3D grid,

$$\boxed{\delta_{pc}(\mathbf{R}) = \frac{10^6}{12\pi R^3}\left[3\hat{\mathbf{R}}^{\mathrm{T}}\chi\hat{\mathbf{R}} - \mathrm{Tr}\,\chi\right]}.$$

It is not an electron-density cube and it does not contain the actual finite-distance ReSpect HFC tensor at each spatial point. NumPy vectorization (`meshgrid`, `einsum`) is used instead of nested Python loops for speed.

9 The main function: execution from start to finish

```
# ------------------------------ MAIN ----------------------------------------
def main() -> None:
    gt = resolve_input_file(S.gt_file).read_text(errors="ignore")
    hfs = resolve_input_file(S.hfs_file).read_text(errors="ignore")
    atoms, gp, hfcc = parse_geometry(gt), parse_g_tensor(gt), parse_hfcc(hfs)
    c_scale, c_now = parse_c_adjust(gt)
    go = parse_gauge_origin(gt)

    g = gp["total"]
    chi = susceptibility(g)
    center = atoms[S.center_atom][1]

    pg = principal_g(g)
    print(f"g_iso={iso(g):.6f} g_principal={pg[0]:.6f},{pg[1]:.6f},{pg[2]:.6f}")
    if go is not None:
        print(f"GO[A]={go[0]:.4f},{go[1]:.4f},{go[2]:.4f}")
    if c_scale is not None:
        print(f"c_scale={c_scale:g} c={c_now:g} a.u." + (" [adjusted]" if abs(c_scale-1) > 1e-12 else ""))

    biggest = max(hfcc, key=lambda i: abs(iso(hfcc[i]["A"])) if "A" in hfcc[i] else 0)
    if biggest != S.center_atom:
        print(f"WARNING: largest |Aiso| is atom #{biggest}; center_atom={S.center_atom}")

    rows, mats, sigmas, rkb_rows, crosses = [], {}, {}, [], {}
    for idx in sorted(hfcc):
        if idx not in atoms or "A" not in hfcc[idx] or "gN" not in hfcc[idx]:
            continue

        el, xyz = atoms[idx]
        rec, gN = hfcc[idx], hfcc[idx]["gN"]
        R = xyz - center
        rn = float(np.linalg.norm(R))
        label = "center" if rn < 1e-9 else "near" if rn < S.near_A else "mid" if rn < S.far_A else "far"

        A = rec["A"]
        dA = {k: shift_ppm(g, rec[k], gN) for k in ("FC", "PSO", "SD", "R1", "R2") if k in rec}
        dg = {k: shift_ppm(gp[k], A, gN) for k in ("SZ", "OZ", "Rel") if k in gp}
        d_total = shift_ppm(g, A, gN)
        d_nonFC = shift_ppm(g, A-rec["FC"], gN) if "FC" in rec else float("nan")
        d_iso, d_ani = shift_iso_ani(g, A, gN)
        A_pda = pda_A_MHz(g, R, gN)
        d_pda = shift_ppm(g, A_pda, gN) if rn > 1e-9 else float("nan")
```

```
        d_ld = ld_shift_ppm(chi, R)

        M = g @ A.T
        mats[idx] = M
        sigmas[idx] = shielding_tensor_ppm(g, A, gN)
        mflat = {f"gAT_{a}{b}": float(M[i, j]) for i, a in enumerate("xyz") for j, b in enumerate("xyz")}
```

The first half of `main()` reads and parses the files, constructs the total g, susceptibility and centre, then loops over every nucleus. Inside that loop it calculates the total shift, nonFC shift, iso/ani split, PDA/LD quantities, raw gA^{T} matrix, full shielding matrix, RKB residuals and the 15 cross terms.

The nonFC scalar is computed directly from

$$A_{\mathrm{nonFC}} = A - A_{\mathrm{FC}},$$

not by subtracting two already-rounded printed scalar shifts. This is numerically cleaner.

```
        rk = rkb_check(gp, rec, R, gN)
        rkflat = {f"rkb_{k}_rel": (rk[f"{k}_rel"] if rk else float("nan")) for k in ("SD", "PSO", "REL", "COMB")}
        if rk:
            rk.update(Atom=idx, El=el, label=label)
            rkb_rows.append(rk)

        cr = cross_terms(gp, rec, gN)
        crosses[idx] = cr
        crflat = {f"cross_{gn}_{an}_ppm": v for (gn, an), v in cr.items()}

        rows.append({
            "Atom": idx, "El": el, "R_A": rn, "label": label, "Aiso_MHz": iso(A),
            "d_total": d_total, "d_nonFC": d_nonFC, "d_iso": d_iso, "d_ani": d_ani,
            **{f"d_{k}": v for k, v in dA.items()},
            **{f"d_g{k}": v for k, v in dg.items()},
            "d_PDA": d_pda, "d_LD": d_ld,
            **mflat, **rkflat, **crflat
        })

    center_row = next((r for r in rows if r["label"] == "center"), rows[0])
    i0 = center_row["Atom"]
    checks = {
        "SI/AU": abs(shift_ppm_au(g, hfcc[i0]["A"], hfcc[i0]["gN"]) - shift_ppm(g, hfcc[i0]["A"], hfcc[i0]["gN"])),
        "PDA/LD": max(abs(r["d_PDA"]-r["d_LD"]) for r in rows if r["R_A"] > 1e-9),
        "iso+ani": max(abs(r["d_iso"]+r["d_ani"]-r["d_total"]) for r in rows),
        "total=FC+nonFC": max(abs(r["d_total"]-r["d_FC"]-r["d_nonFC"]) for r in rows if "d_FC" in r),
        "nonFC=sum(parts)": max(abs(r["d_nonFC"]-sum(r[f"d_{k}"] for k in ("PSO","SD","R1","R2"))) for r in rows
                                if all(f"d_{k}" in r for k in ("PSO","SD","R1","R2"))),
        "sum(15)": max(abs(sum(crosses[r["Atom"]].values())-r["d_total"]) for r in rows if crosses[r["Atom"]]),
    }
    print("checks[ppm]: " + " ".join(f"{k}={v:.2e}" for k, v in checks.items()))

    print_tables12(rows)
    print_gAT(rows, mats, sigmas)
    print_rkb(rkb_rows)
    print_cross(rows, crosses)

    if S.write_txt:
        txt_path = write_text_report(rows, mats, sigmas, rkb_rows, crosses,
                                     g, go, c_scale, c_now, checks)
        print(f"\nTXT: {txt_path}")

    if S.write_csv:
        csv_path = write_csv(rows)
        print(f"\nCSV: {csv_path}")
    if S.write_cube:
        info = write_cube(atoms, center, chi)
        print(f"CUBE: {info['path']} grid={info['nx']}x{info['ny']}x{info['nz']} "
              f"range={info['min']:.3g}..{info['max']:.3g} ppm")


if __name__ == "__main__":
    main()
```

The second half stores one dictionary per nucleus, computes the self-checks, prints all sections, then writes the requested files.

Meaning of the six checks

$$\begin{aligned}
\text{SI/AU}: &\quad \delta_{SI} \simeq \delta_{au},\\
\text{PDA/LD}: &\quad \delta_{PDA} \simeq \delta_{LD},\\
\text{iso} + \text{ani}: &\quad \delta_{iso} + \delta_{ani} \simeq \delta_{total},\\
\text{total} = \text{FC} + \text{nonFC}: &\quad \delta_{FC} + \delta_{nonFC} \simeq \delta_{total},\\
\text{nonFC} = \text{sum(parts)}: &\quad \delta_{PSO} + \delta_{SD} + \delta_{R1} + \delta_{R2} \simeq \delta_{nonFC},\\
\text{sum(15)}: &\quad \sum_{ij} \delta(g_i, A_j) \simeq \delta_{total}.
\end{aligned}$$

These are mainly algebra/unit consistency checks. Tiny nonzero differences arise from floating-point arithmetic and the finite precision of tensors printed by ReSpect.

10 How to read the current numerical results

Running the attached code with the current ReSpect files gives

$$g_{iso} = 2.002996, \qquad (g_1, g_2, g_3) = (2.000797, 2.002397, 2.005795),$$

with adjusted c reported as $c_{scale} = 0.5$ and $c = 68.518$ a.u. The program does not apply a posteriori rescaling; the modified c has already affected the ReSpect tensors $g(c)$ and $A(c)$.

The numerical checks are

SI/AU	6.81×10^{-8} ppm
PDA/LD	3.24×10^{-14} ppm
$iso + ani$	1.82×10^{-12} ppm
$total = FC + nonFC$	2.10×10^{-12} ppm
$nonFC = sum(parts)$	7.74×10^{-8} ppm
$sum(15)$	5.16×10^{-7} ppm

so the implemented algebra is numerically self-consistent.

Representative Table-1 values are:

Atom	R/Å	δ_{total}	δ_{nonFC}	δ_{PDA}	δ_{LD}
C1	0.000	3.449×10^4	−286	–	–
C3	2.483	9002	−69.2	−0.259	−0.259
C8	8.672	−234.8	2.64	−0.00645	−0.00645
H15	8.819	38.65	−0.0515	−0.00647	−0.00647
C19	11.146	−169.9	1.87	−0.00301	−0.00301
H21	12.079	34.53	−0.0521	−0.00230	−0.00230

Two points are important. First, δ_{total} is often dominated by the FC contribution; therefore a large total Curie shift does not mean a large long-distance PCS. Second, the equality $\delta_{PDA} = \delta_{LD}$ only proves that the two point-dipole implementations are consistent with one another. To ask whether the real finite-distance nonFC response is already point-dipolar, compare δ_{nonFC} with δ_{LD} and inspect the tensor diagnostics.

For example, atom C19 has

$$\delta_{nonFC} = 1.87 \text{ ppm}, \qquad \delta_{LD} = -0.00301 \text{ ppm},$$

so geometric distance alone is not sufficient to conclude that the actual nonFC contribution has reached the ideal point-dipole limit for this calculation.

The shielding matrix output should be read separately from δ. For atom C2 the program prints $\sigma_{iso} = +578.677$ ppm and then

$$\delta_{total} = -578.7 \text{ ppm},$$

which illustrates directly the sign convention $\delta = -\sigma_{iso}$.

11 What to edit for a new calculation

For a normal new system, edit only `Settings`: filenames, T, S, centre atom and desired output options. If the ReSpect text format changes, edit `parse_g_tensor()` or the `tags` dictionary in `parse_hfcc()`. If a new A contribution is added and should appear in Table 4, it must be added consistently to the parser, the `dA` tuple, `cross_terms()` and `print_cross()`.

A useful rule is:

- change **SETTINGS** for a new calculation;
- change **PARSING** only if the ReSpect output format changes;
- change **PHYSICS** only if the theoretical model/equations change;
- change **OUTPUT** only if a different presentation is wanted.

12 Compact interpretation guide

Quantity	**Read it as**
gA^{T}	Raw tensor product before the Curie prefactor.
$10^6\boldsymbol{\sigma}^{Curie}$	Full Curie shielding tensor printed on the ppm scale.
`d_total`	Scalar total Curie shift δ_{Curie} in ppm.
`d_nonFC`	Scalar shift from $A - A_{FC}$.
`d_PDA`, `d_LD`	Same ideal point-dipole scalar result obtained by two routes.
Table 2	Bilinear decomposition by ReSpect A parts and g parts.
Table 3	Tensor-distance diagnostics; SD/SZ is the cleanest direct LD comparison.
Table 4	All 15 numerical $g_i \times A_j$ contributions; sum gives `d_total`.
TXT	Human-readable complete report.
CSV	Machine-readable data for plotting/analysis.
CUBE	3D ideal long-distance PCS field.

Bottom line. The code is intentionally built around one reusable bilinear Curie-shift function. Parsing supplies the tensors, the physics functions transform them, the checks verify consistency, and the output functions present the same results in several complementary forms. This separation makes the code short enough to audit and easy to modify without mixing file handling, physics and presentation.